Sorting: evaluation, improvements, and comparisons

Jianguo Lu

University of Windsor

November 20, 2025

6 5 3 1 8 7 2 4

Overview

What is sorting

- Input
  - An array A of data records
  - A Key value for each data record
  - A comparison function
- Output
  - Permutation of A such that
  - if $i < j$, then $A[i] < A[j]$

Why sort

- Sorting algorithms are among the most frequently used algorithms in computer science

  Searching : Allows binary search of an N-element array in $O(\log N)$ time

  Top elements : Allows $O(1)$ time access to k-th largest element in the array for any k

  Detect duplicates : Allows easy detection of any duplicates

  ...

Evaluation of sorting algorithms

- Time

Evaluation of sorting algorithms

- Time
- Space

Evaluation of sorting algorithms

- Time
- Space
- Stability

Evaluation of sorting algorithms

- Time
- Space
- Stability
- Data dependency:
  - some algorithms are good for *partially* sorted data
  - some are good if the keys are within a certain ranges. e.g., bucket sort.

Evaluation of sorting algorithms

- Time
- Space
- Stability
- Data dependency:
  - some algorithms are good for *partially* sorted data
  - some are good if the keys are within a certain ranges. e.g., bucket sort.
- Application dependency: e.g., we only want to have the top few.

Evaluation of sorting algorithms

- Time
- Space
- Stability
- Data dependency:
  - some algorithms are good for *partially* sorted data
  - some are good if the keys are within a certain ranges. e.g., bucket sort.
- Application dependency: e.g., we only want to have the top few.
- External: data too large to fit in memory

Evaluation of sorting algorithms

- Time
- Space
- Stability
- Data dependency:
  - some algorithms are good for *partially* sorted data
  - some are good if the keys are within a certain ranges. e.g., bucket sort.
- Application dependency: e.g., we only want to have the top few.
- External: data too large to fit in memory
- Parallel algorithms: data too large to run on one computer

Time: How fast is the algorithm

- The definition of a sorted array A says that for any $i < j$, $A[i] < A[j]$
- Need to at least check on each element at the very minimum, I.e., at least O(N)
- Could end up checking each element against every other element, which is $O(N^2)$.
- The big question is: How close to O(N) can you get?

Space

How much space does the sorting algorithm require in order to sort the collection of items?

- Is copying needed?
- In-place sorting: no copying. O(1) additional space
- some where in between for "temporary", e.g. $O(\log n)$ space
- External memory sorting: data so large that does not fit in memory

In-place algorithms

An algorithm is *in-place* if it uses only a small amount of memory in addition to that needed for the original input.

Whether the following algorithms are in-place sorting?

- Merge sort?

Space

How much space does the sorting algorithm require in order to sort the collection of items?

- Is copying needed?
- In-place sorting: no copying. O(1) additional space
- some where in between for "temporary", e.g. $O(\log n)$ space
- External memory sorting: data so large that does not fit in memory

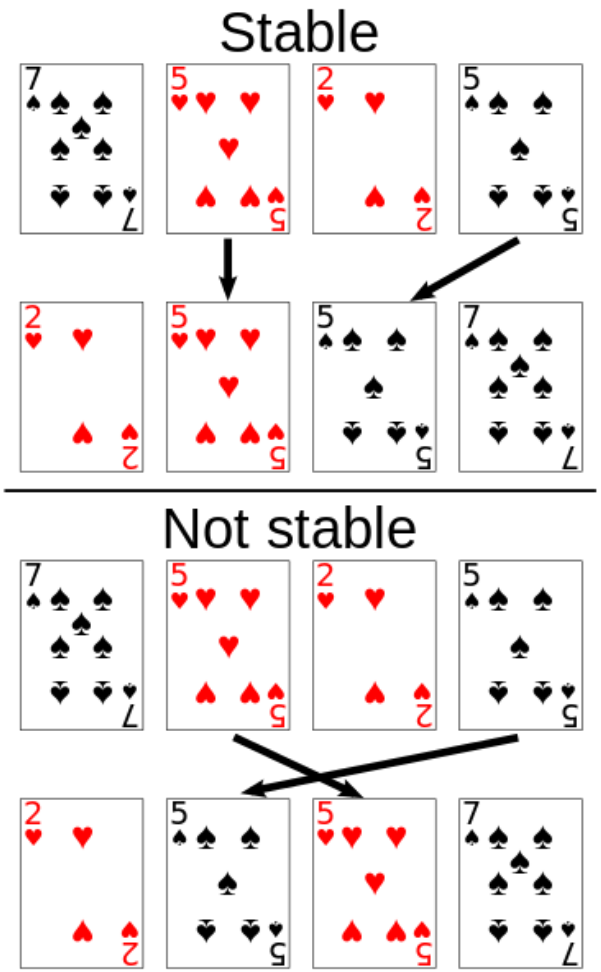
In-place algorithms

An algorithm is *in-place* if it uses only a small amount of memory in addition to that needed for the original input.

Whether the following algorithms are in-place sorting?

- Merge sort?
- Insertion sort?

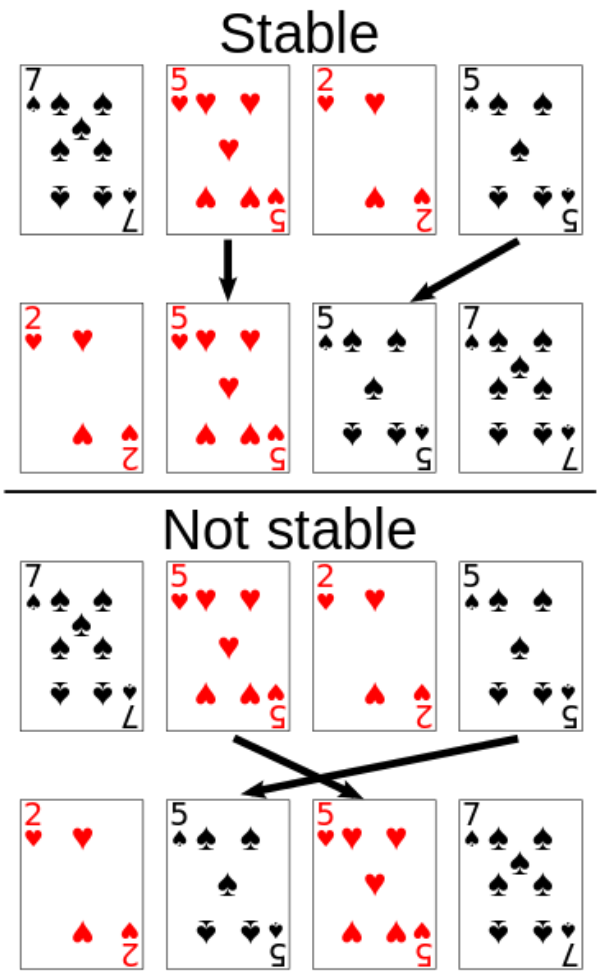
Stability



Does it rearrange the order of input data records which have the same key value (duplicates)?

- E.g. Phone book sorted by name. Now sort by county. Is the list still sorted by name within each county?
- Extremely important property for databases

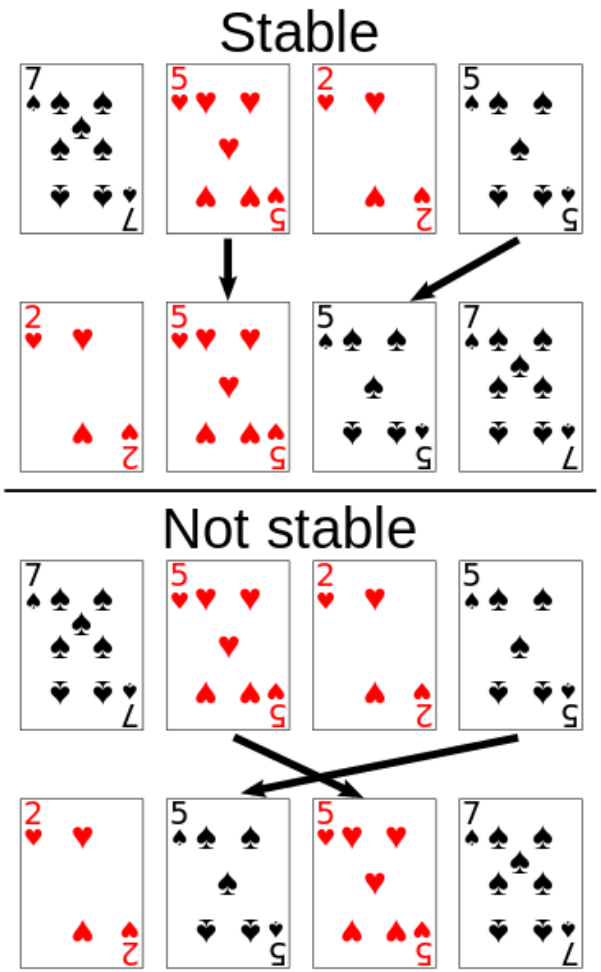
Stability



Does it rearrange the order of input data records which have the same key value (duplicates)?

- E.g. Phone book sorted by name. Now sort by county. Is the list still sorted by name within each county?
- Extremely important property for databases
- Whether insertion sort is stable?

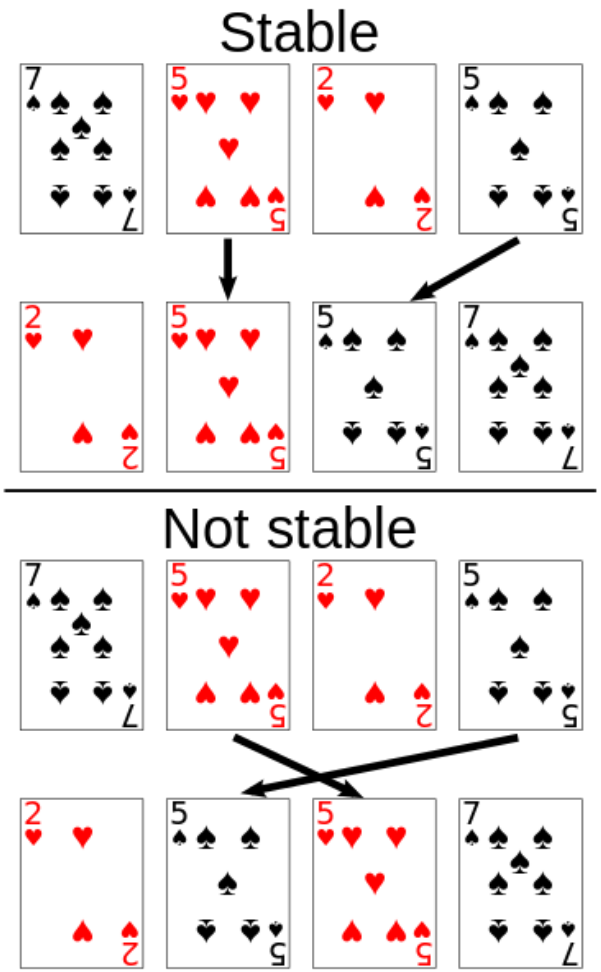
Stability



Does it rearrange the order of input data records which have the same key value (duplicates)?

- E.g. Phone book sorted by name. Now sort by county. Is the list still sorted by name within each county?
- Extremely important property for databases
- Whether insertion sort is stable?
- How about merge sort?

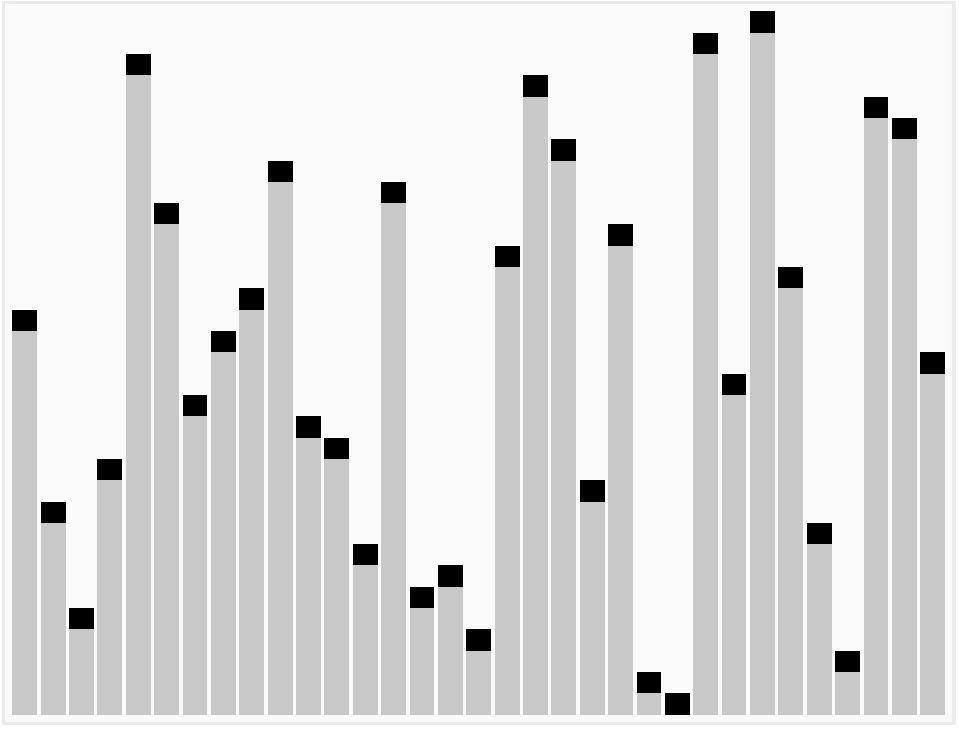
Stability



Does it rearrange the order of input data records which have the same key value (duplicates)?

- E.g. Phone book sorted by name. Now sort by county. Is the list still sorted by name within each county?
- Extremely important property for databases
- Whether insertion sort is stable?
- How about merge sort?
- How about heap sort?

Quick sort



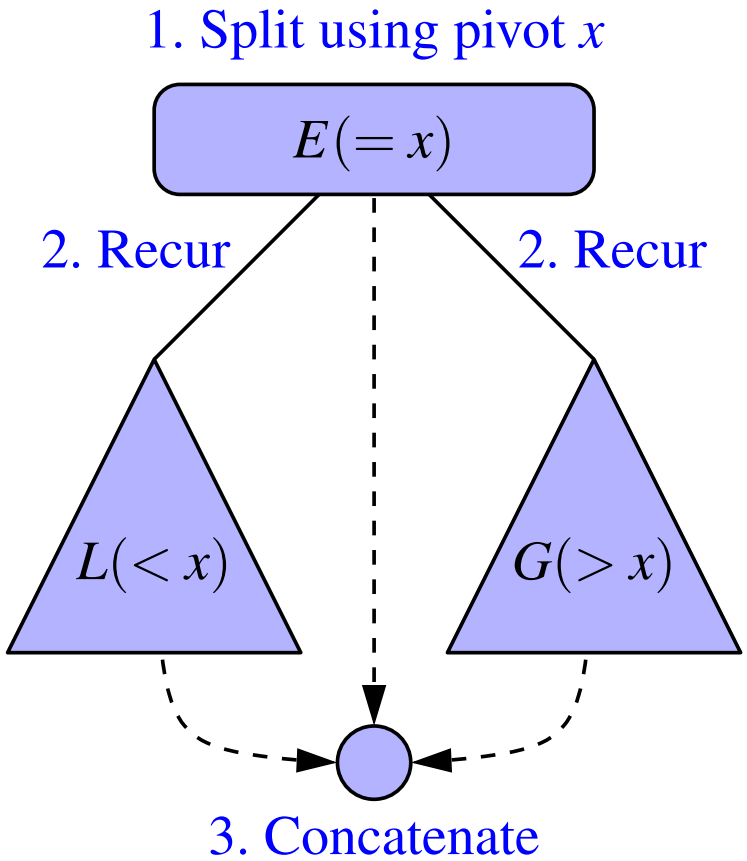
Overall idea: Divide-and Conquer

Quick-sort is a randomized sorting algorithm based on the divide-and-conquer paradigm

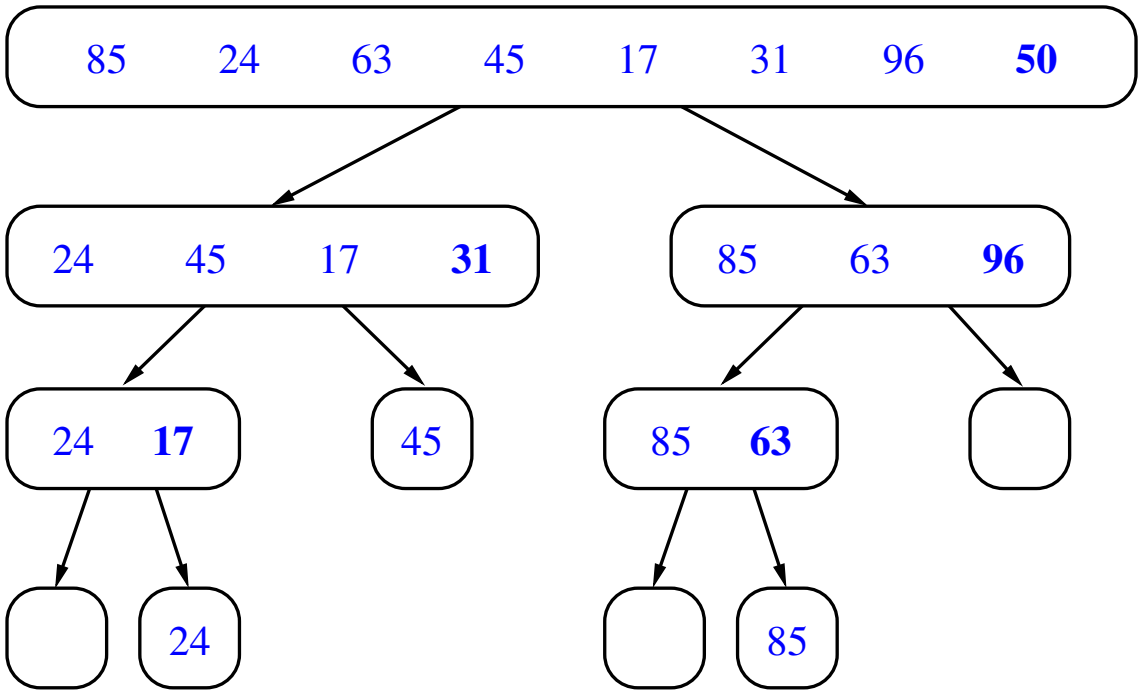
Divide pick a random element x (called pivot) and partition S into
- L elements less than x
- E elements equal x
- G elements greater than x

Conquer Recursively sort L and G

Combine join L, E and G

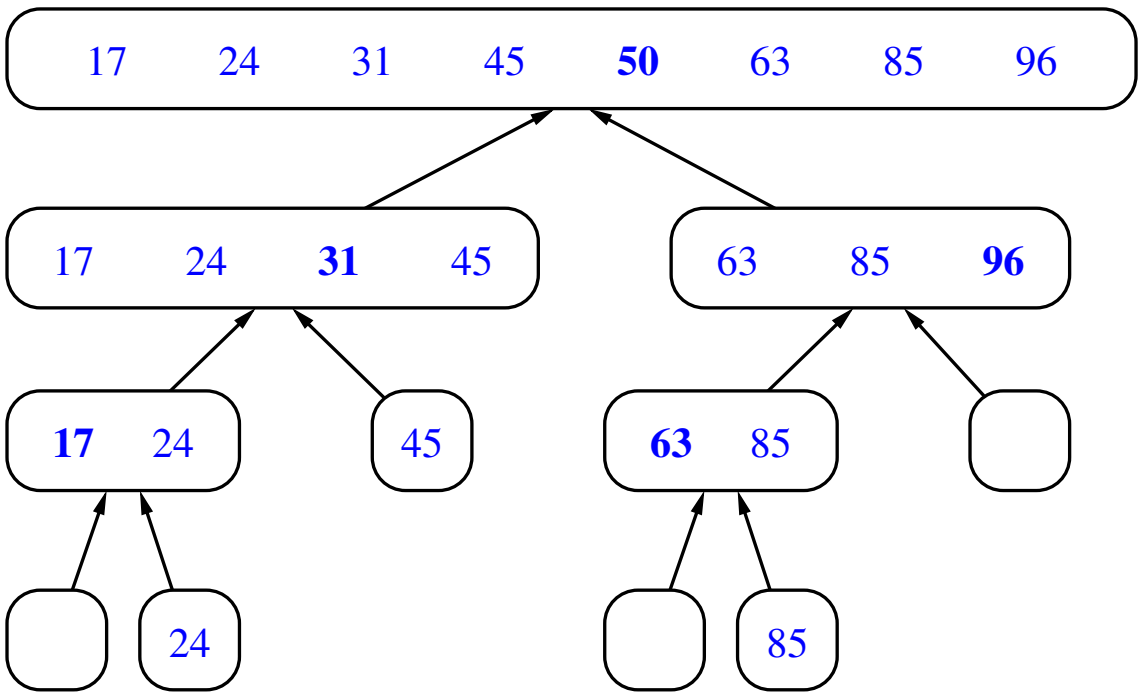


Quick sort example: divide



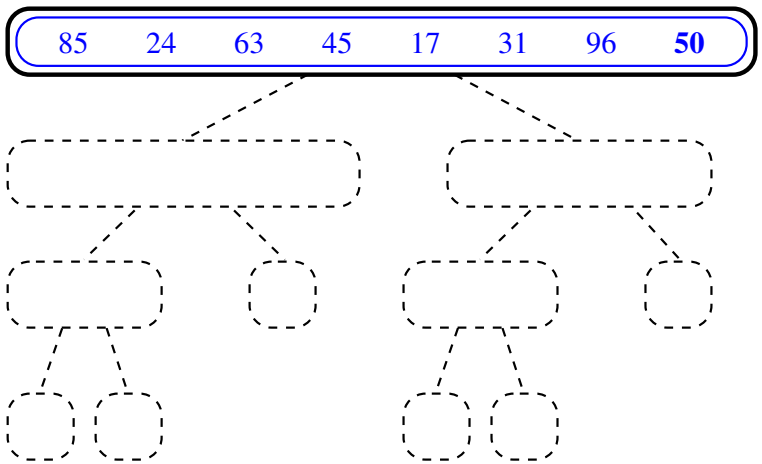
Divide process

Combine

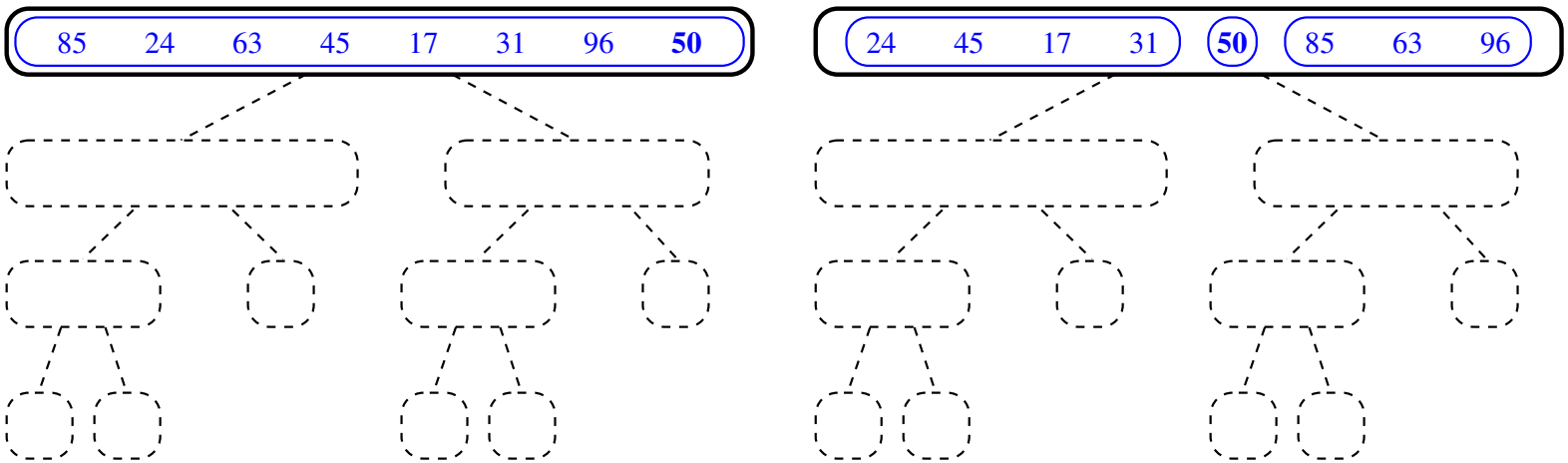


Combination process

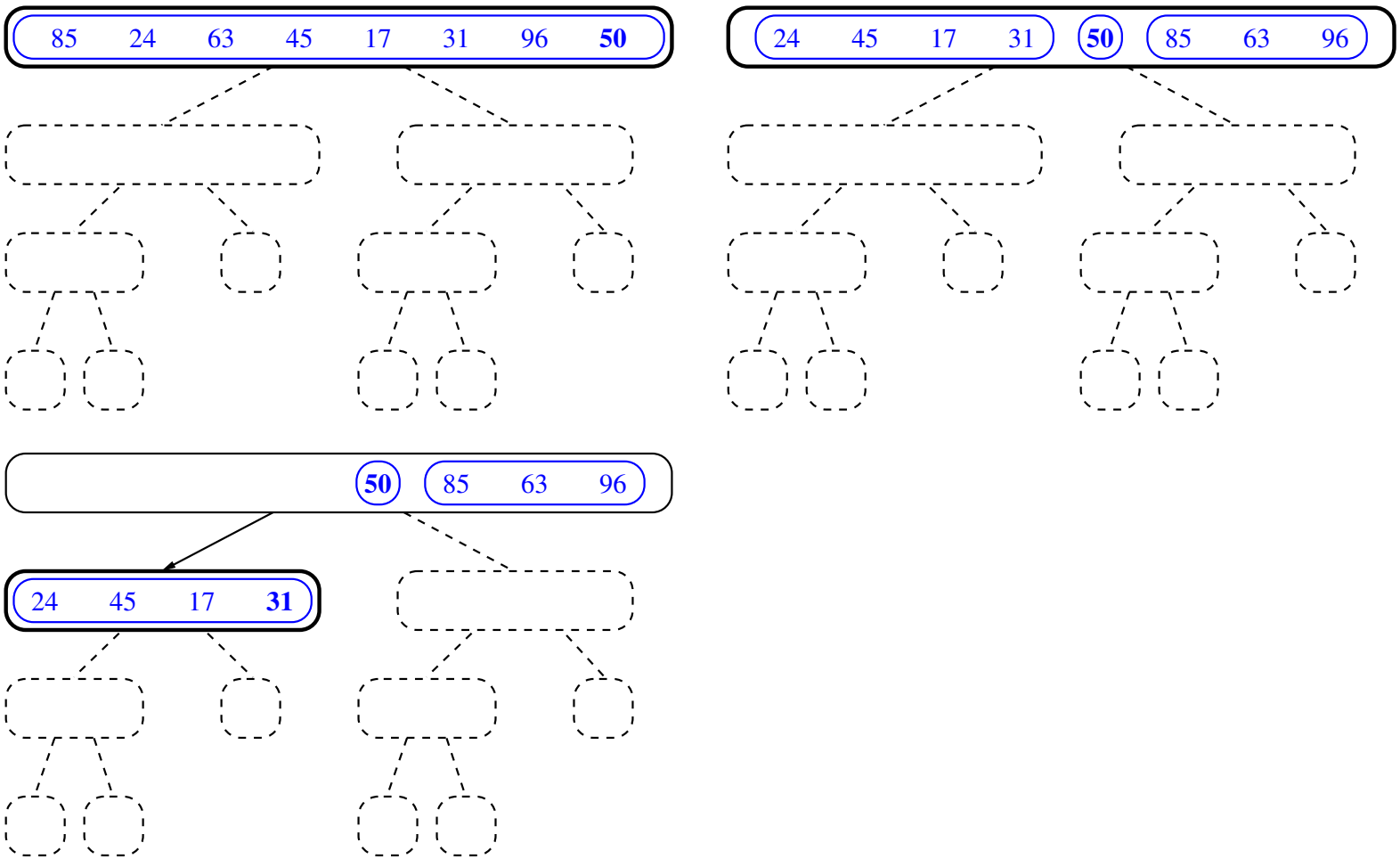
Quick sort step-by-step: recursive calls



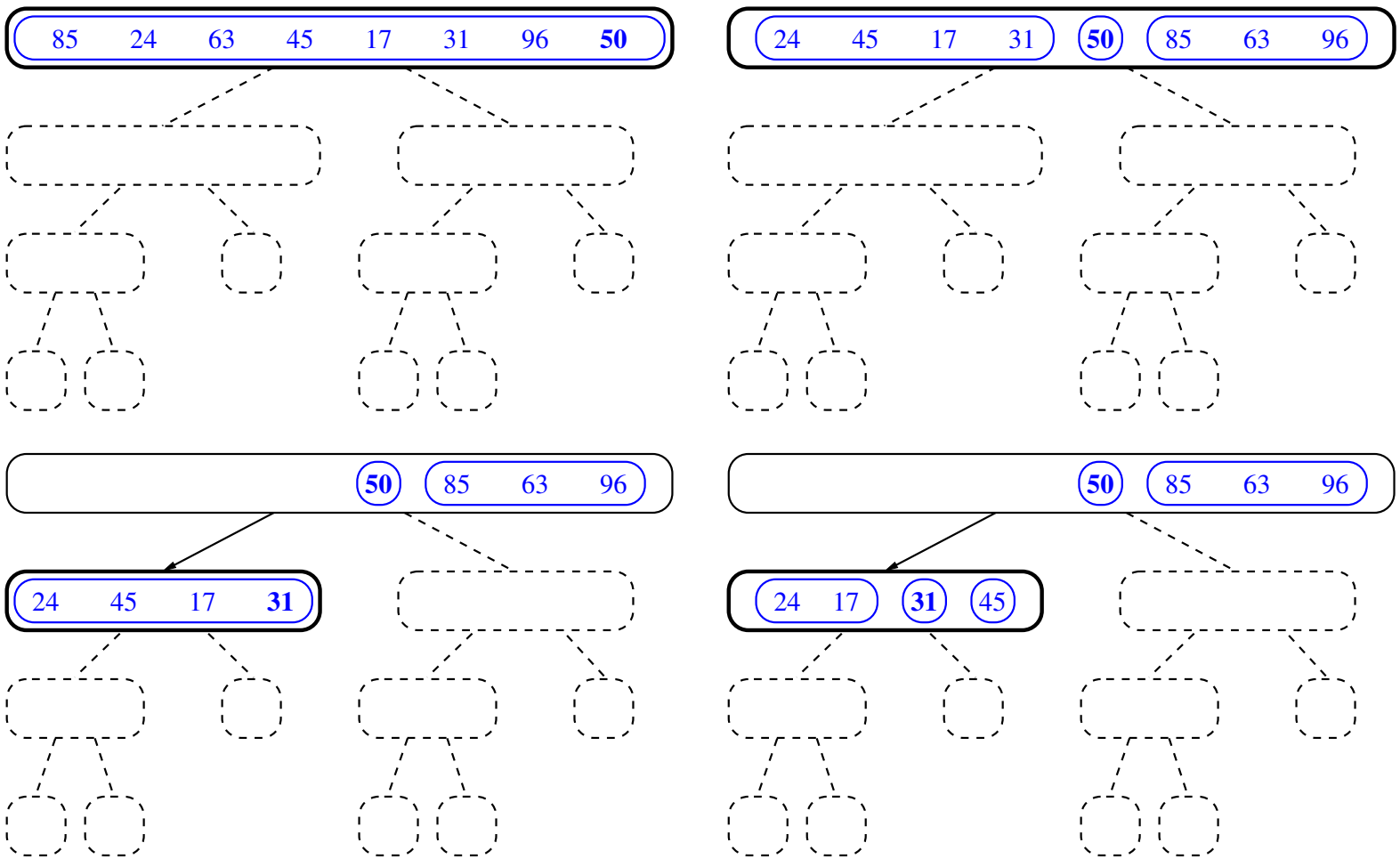
Quick sort step-by-step: recursive calls



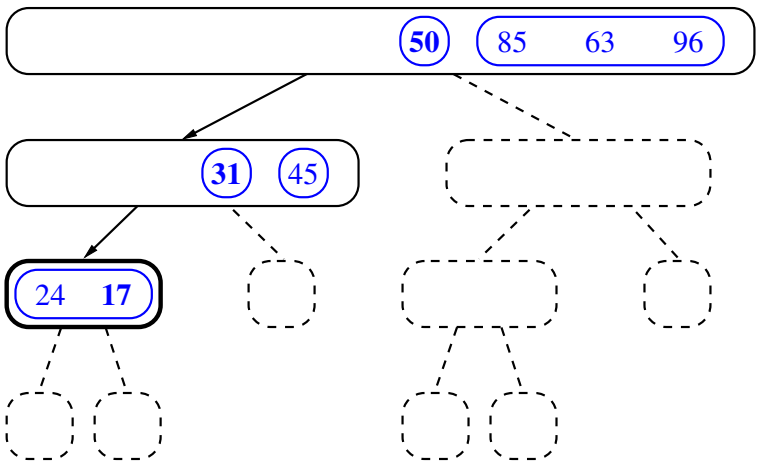
Quick sort step-by-step: recursive calls



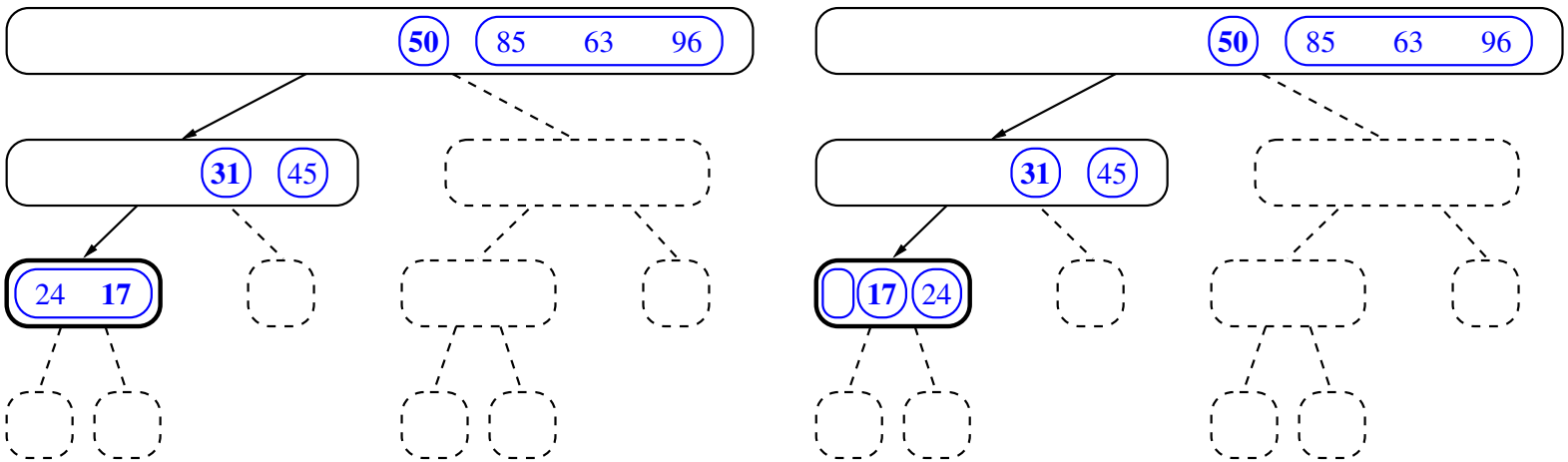
Quick sort step-by-step: recursive calls



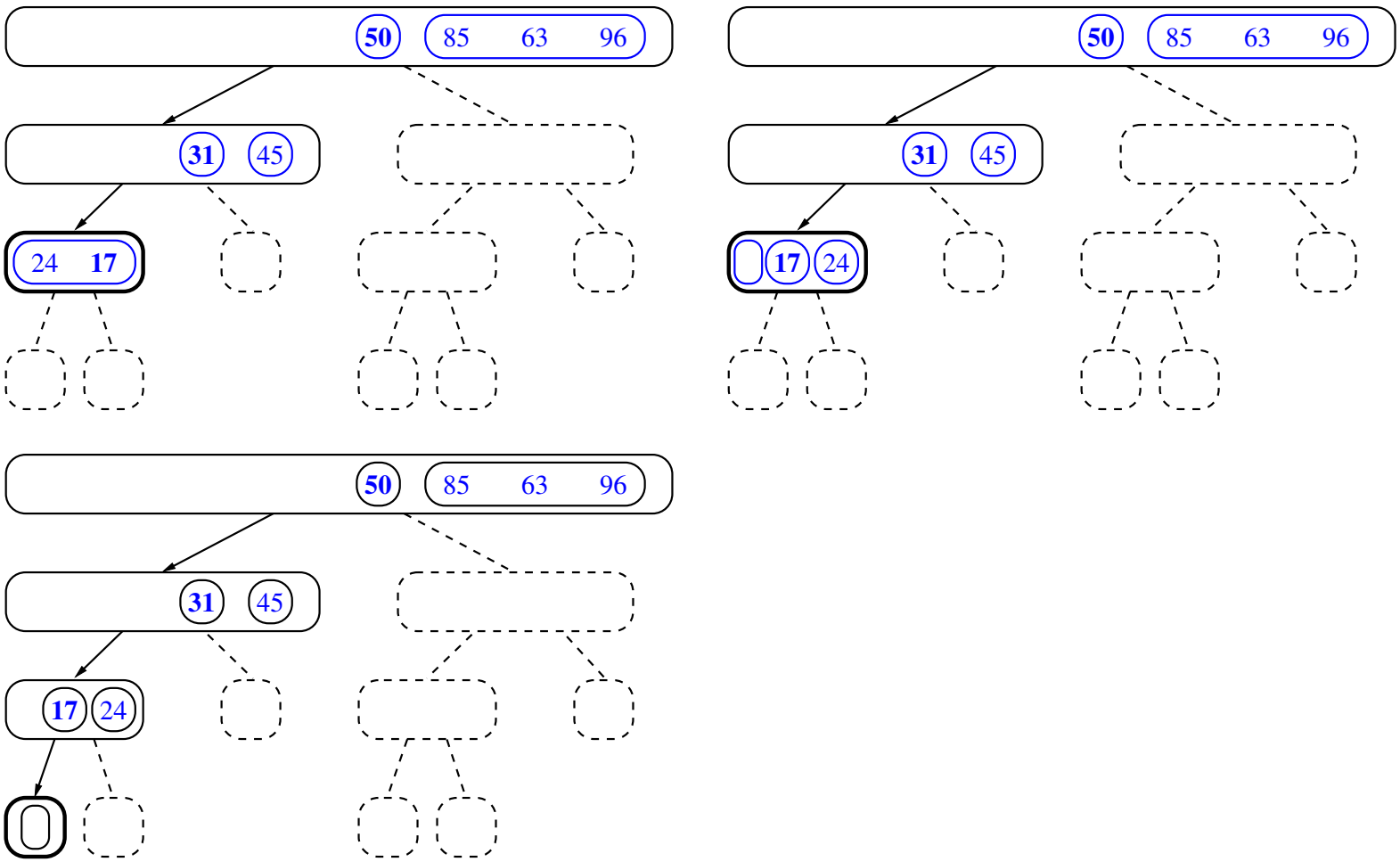
Recursive calls down to the base case



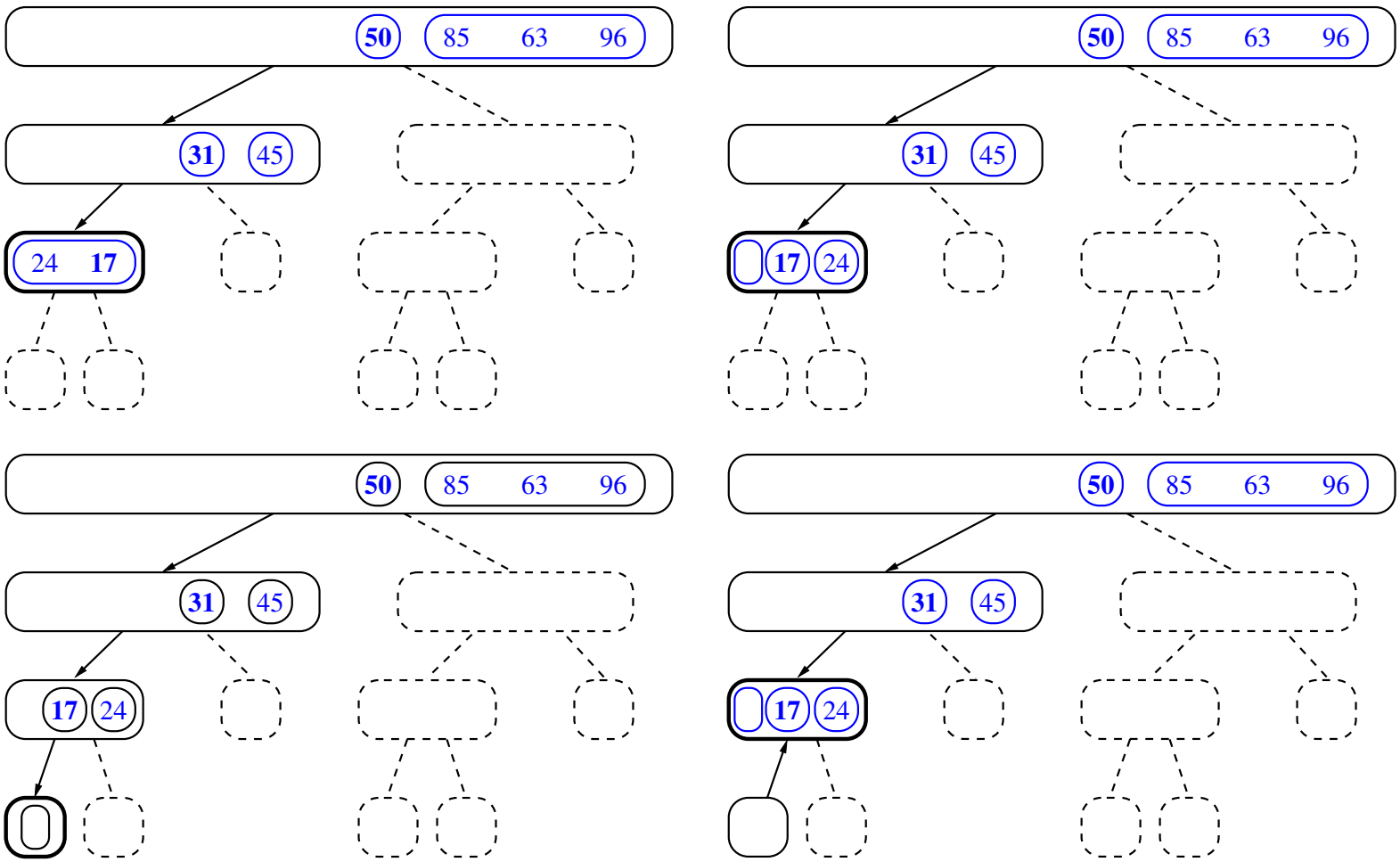
Recursive calls down to the base case



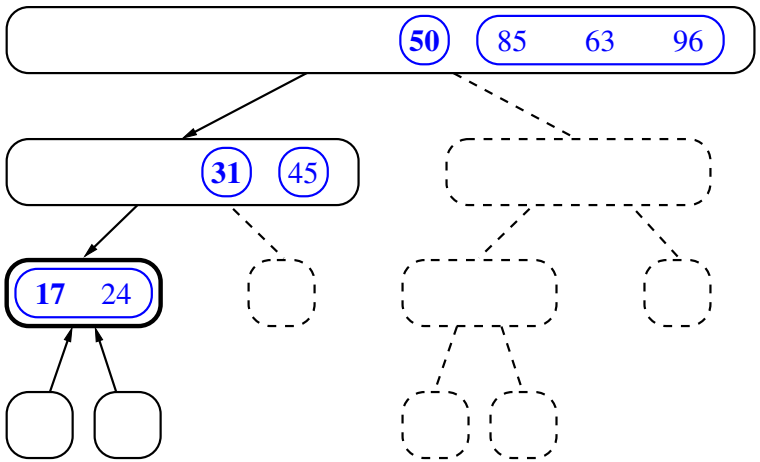
Recursive calls down to the base case



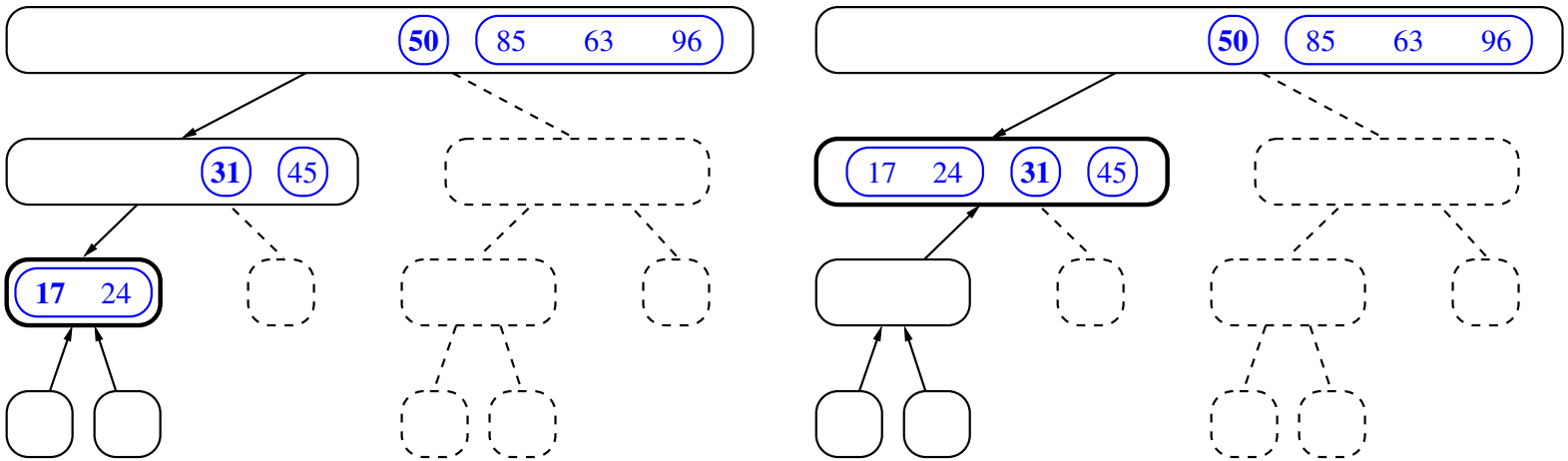
Recursive calls down to the base case



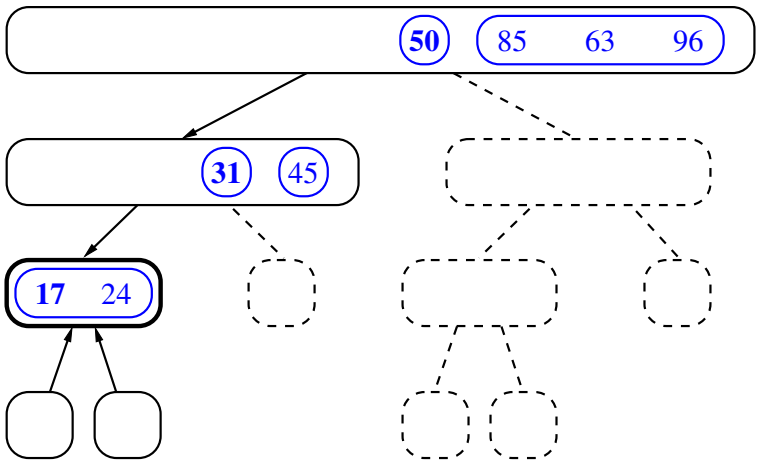
Return of recursive calls, then call (45) branch

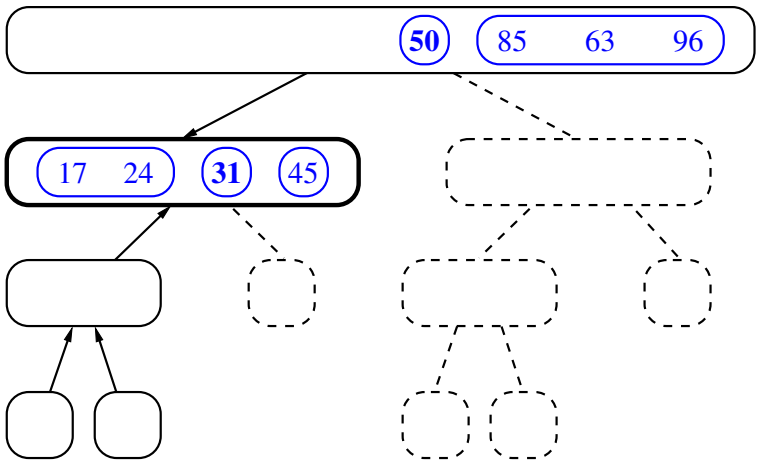


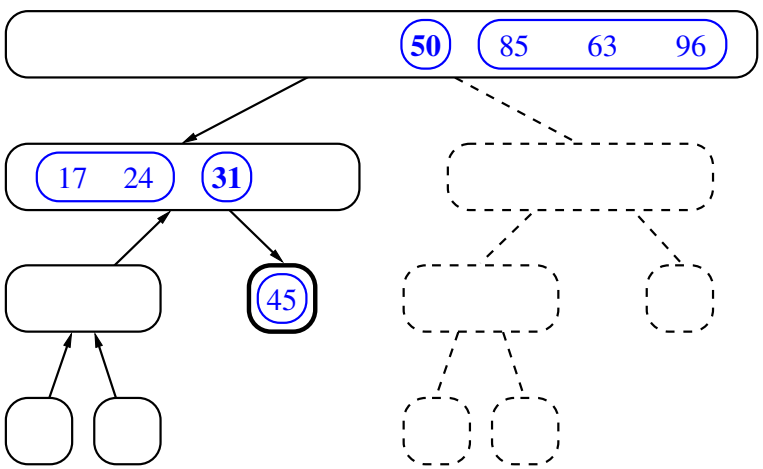
Return of recursive calls, then call (45) branch



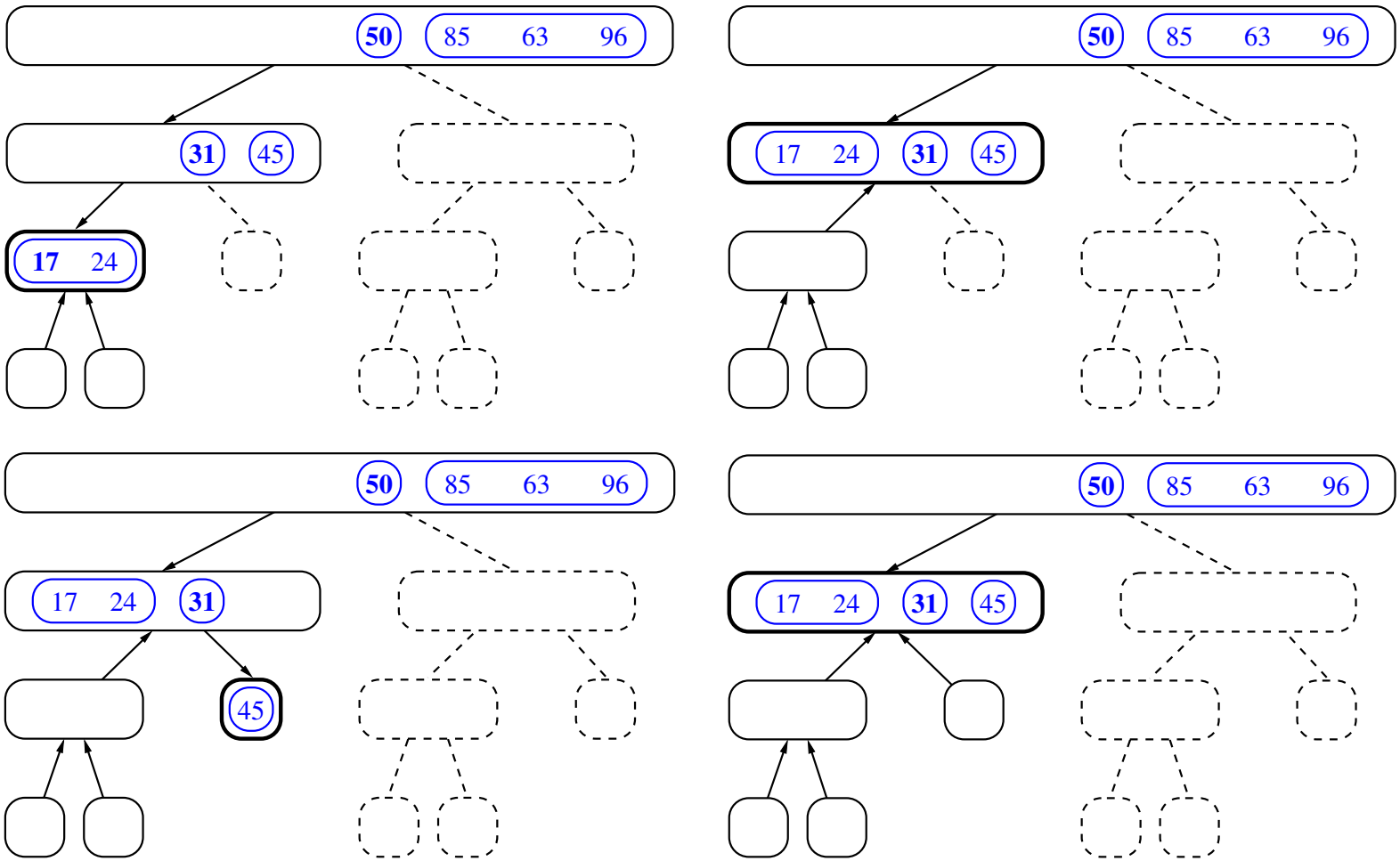
Return of recursive calls, then call (45) branch



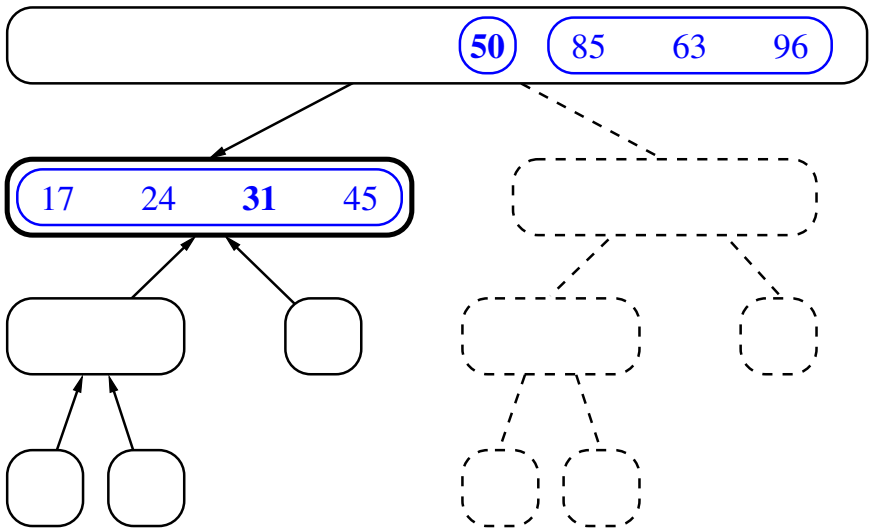




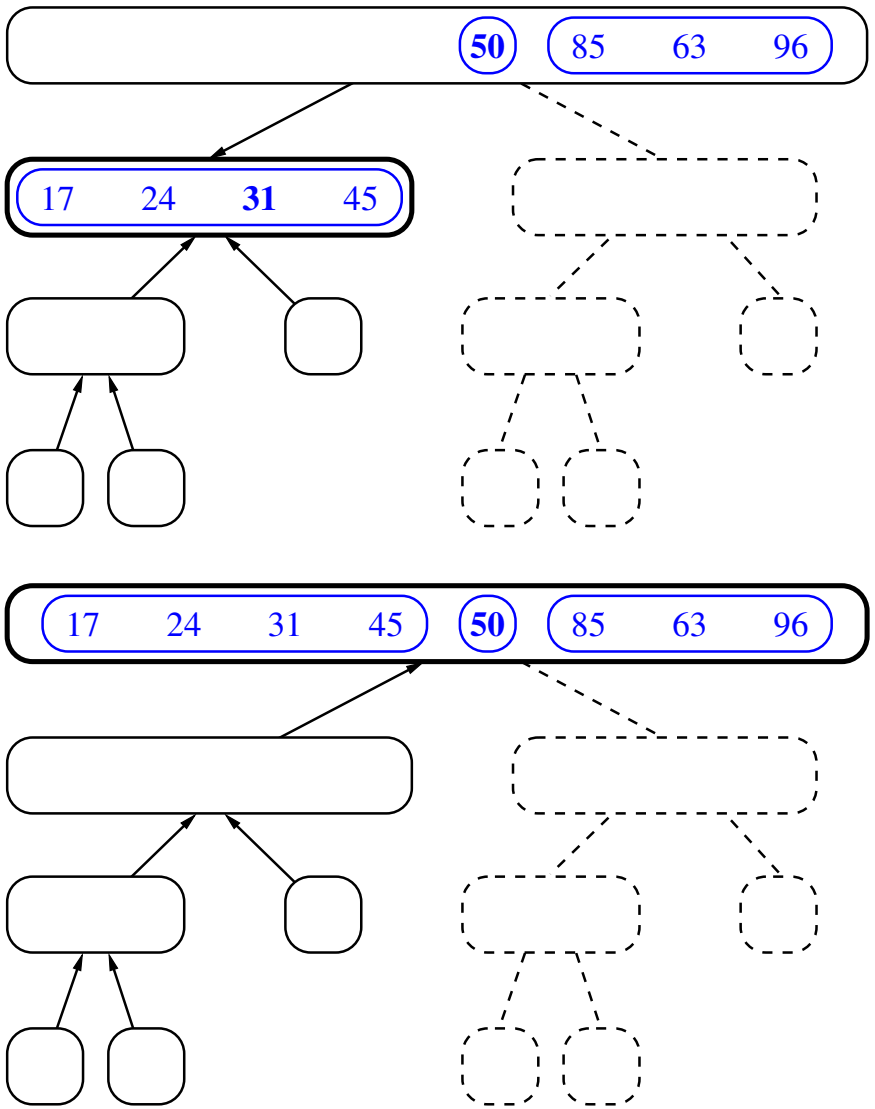
Return of recursive calls, then call (45) branch



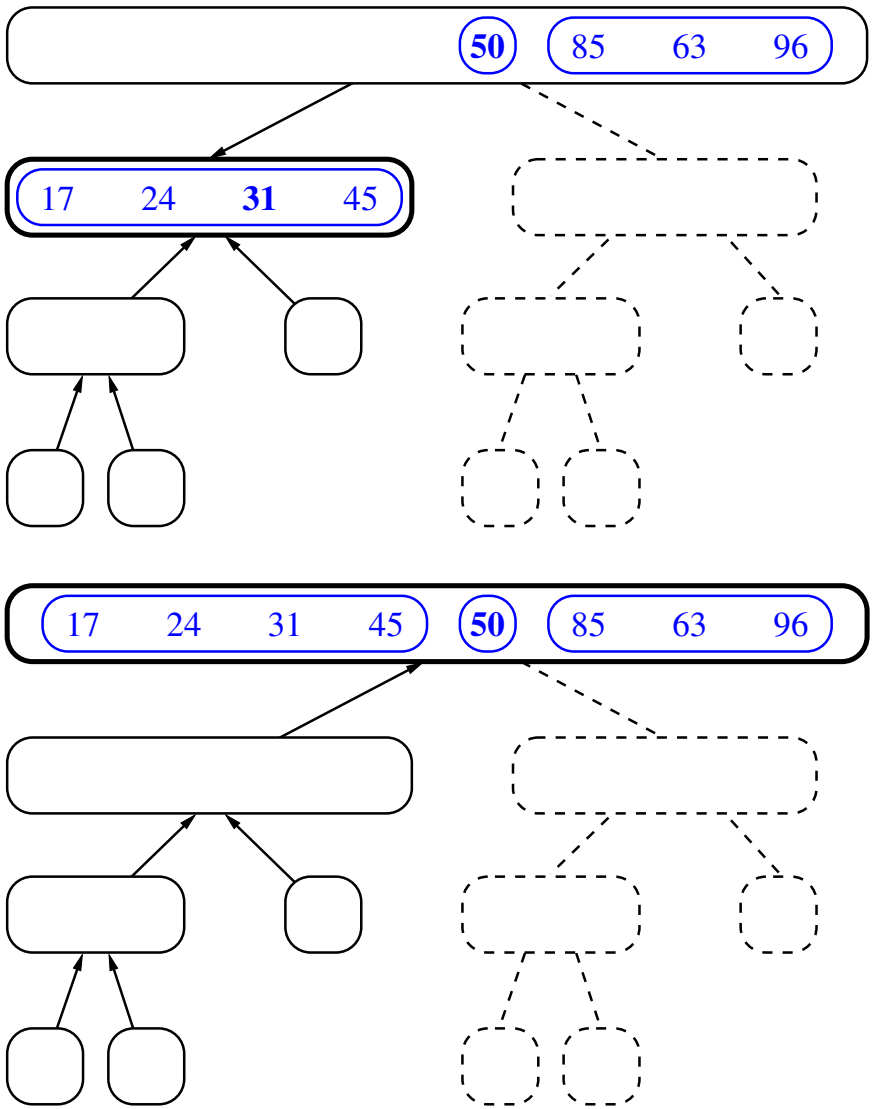
Return of the left branch, do the same for the right branch



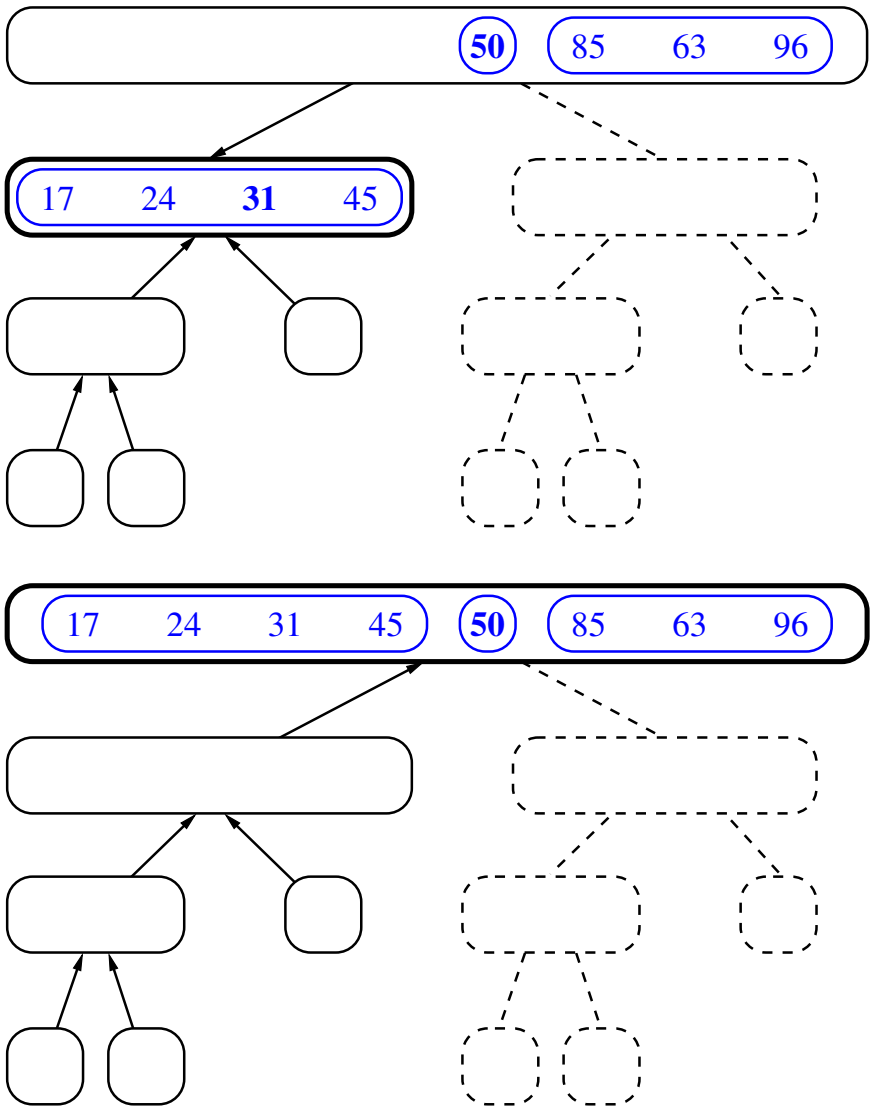
Return of the left branch, do the same for the right branch



Return of the left branch, do the same for the right branch



Return of the left branch, do the same for the right branch



Implementation on Queue (bad one)

Algorithm 1: quickSortBad(Queue Q)

```
4 (L, E, G)=partition(Q);
5 quickSort(L, comp);
6 quickSort(G, comp);
```

Implementation on Queue (bad one)

Algorithm 2: quickSortBad(Queue Q)

```
1 if Q.size<2 then
2 |   return;
3 end
4 (L, E, G)=partition(Q);
5 quickSort(L, comp);
6 quickSort(G, comp);
```

Implementation on Queue (bad one)

Algorithm 3: quickSortBad(Queue Q)

```
1 if Q.size<2 then
2 |   return;
3 end
4 (L, E, G)=partition(Q);
5 quickSort(L, comp);
6 quickSort(G, comp);
7 while (!L.isEmpty()) Q.enqueue(L.dequeue());
8 while (!E.isEmpty()) Q.enqueue(E.dequeue());
9 while (!G.isEmpty()) Q.enqueue(G.dequeue());
```

Queue partition(bad one)

Algorithm 4: (L,E,G) partition(Q)

```
1  pivot =Q.first();
2  L, E, G = new LinkedQueue();
3  while Q not empty do
4  |   nextElement = Q.dequeue();
5  |   switch compare nextElement with pivot do
6  |   |   case nextElement < pivot do
7  |   |   |   L.enqueue(nextElement);
8  |   |   end
9  |   |   case nextElement = pivot do
10 |   |   |   E.enqueue(nextElement);
11 |   |   end
12 |   |   case nextElement > pivot do
13 |   |   |   G.enqueue(nextElement);
14 |   |   end
15 |   end
16 end
```

Performance of partition (queue based)

- Time complexity:
  - remove each element y from Q
  - insert y into L, E or G, depending on the comparison with the pivot x
  - Each insertion and removal is at the beginning or at the end of a sequence, and hence takes $O(1)$ time
  - Thus, the partition step of quick-sort takes $O(n)$ time

Performance of partition (queue based)

- Time complexity:
  - remove each element y from Q
  - insert y into L, E or G, depending on the comparison with the pivot x
  - Each insertion and removal is at the beginning or at the end of a sequence, and hence takes $O(1)$ time
  - Thus, the partition step of quick-sort takes $O(n)$ time
- Space complexity for partition:

Performance of partition (queue based)

- Time complexity:
  - remove each element y from Q
  - insert y into L, E or G, depending on the comparison with the pivot x
  - Each insertion and removal is at the beginning or at the end of a sequence, and hence takes $O(1)$ time
  - Thus, the partition step of quick-sort takes $O(n)$ time
- Space complexity for partition:
  - $O(n)$

Implementation on Queue (bad one) in Java

```
public static <K> void quickSort(Queue<K> S, Comparator<K> comp) {
  int n = S.size();
  if (n < 2) return;
  K pivot = S.first();
  Queue<K> L = new LinkedQueue<>();
  Queue<K> E = new LinkedQueue<>();
  Queue<K> G = new LinkedQueue<>();
  while (!S.isEmpty()) {
    K element = S.dequeue();
    int c = comp.compare(element, pivot);
    if (c < 0) L.enqueue(element);
    else if (c == 0) E.enqueue(element);
    else G.enqueue(element);
  }
  quickSort(L, comp);
  quickSort(G, comp);
  while (!L.isEmpty())       S.enqueue(L.dequeue());
  while (!E.isEmpty())       S.enqueue(E.dequeue());
  while (!G.isEmpty())       S.enqueue(G.dequeue());
}
```

Quick sort on Array

- A better quickSort that does not need an extra array

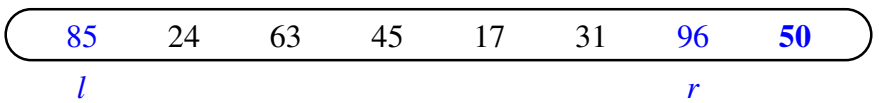
Algorithm 5: quickSortInPlace(A, int low, int high)

```
1 if low ≥ high then
2 |   return;
3 end
4 int p=partition(A, low, high);
5 quickSort(A, low, p-1);
6 quickSort(A, p+1, high);
```

In-place Partition (rewritten from the book)

Algorithm 6: partition(A, l, r)

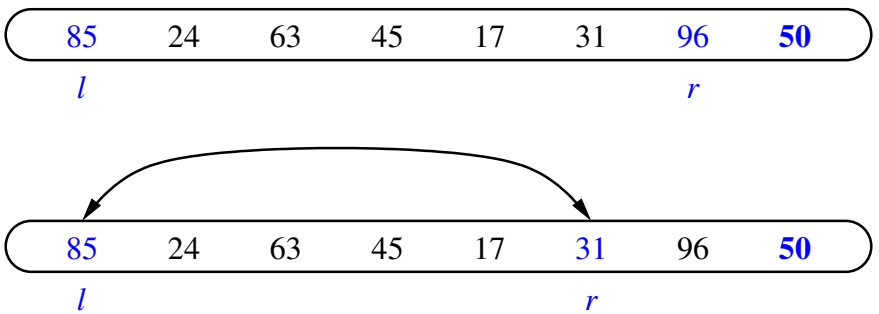
```
1  pivot=A[r];
2  p=r;
3  r=r-1;
4  while l ≤ r do
5  |   while l ≤ r and A[l] < pivot do
6  |   |   l++;
7  |   end
8  |   while l ≤ r and A[r] > pivot do
9  |   |   r- -;
10 |   end
11 |   if l ≤ r then
12 |   |   swap(A[l], A[r]);
13 |   |   l++; r- -;
14 |   end
15 end
16 swap(A[l], A[p]);
17 return l;
```



In-place Partition (rewritten from the book)

Algorithm 7: partition(A, l, r)

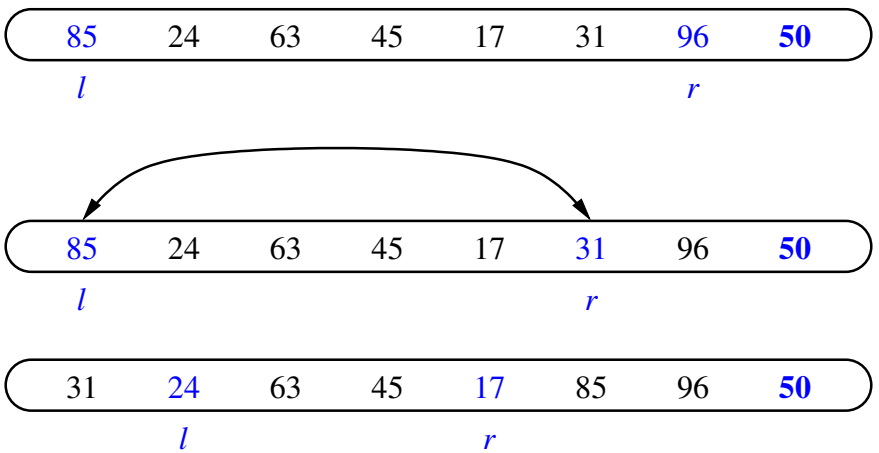
```
1  pivot=A[r];
2  p=r;
3  r=r-1;
4  while l ≤ r do
5  |   while l ≤ r and A[l] < pivot do
6  |   |   l++;
7  |   end
8  |   while l ≤ r and A[r] > pivot do
9  |   |   r- -;
10 |   end
11 |   if l ≤ r then
12 |   |   swap(A[l], A[r]);
13 |   |   l++; r- -;
14 |   end
15 end
16 swap(A[l], A[p]);
17 return l;
```



In-place Partition (rewritten from the book)

Algorithm 8: partition(A, l, r)

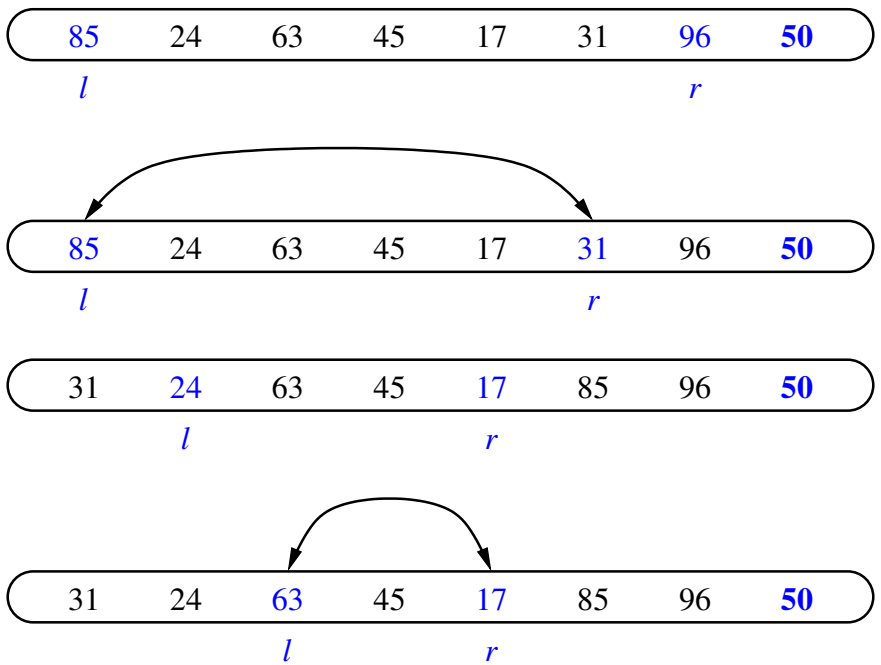
```
1  pivot=A[r];
2  p=r;
3  r=r-1;
4  while l ≤ r do
5  |   while l ≤ r and A[l] < pivot do
6  |   |   l++;
7  |   end
8  |   while l ≤ r and A[r] > pivot do
9  |   |   r- -;
10 |   end
11 |   if l ≤ r then
12 |   |   swap(A[l], A[r]);
13 |   |   l++; r- -;
14 |   end
15 end
16 swap(A[l], A[p]);
17 return l;
```



In-place Partition (rewritten from the book)

Algorithm 9: partition(A, l, r)

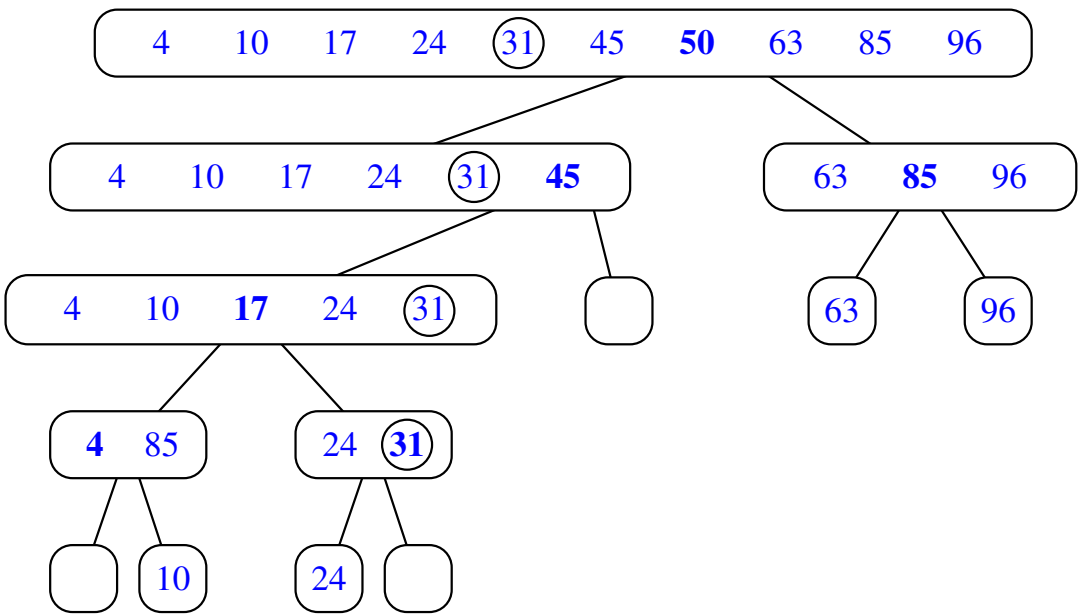
```
1  pivot=A[r];
2  p=r;
3  r=r-1;
4  while l ≤ r do
5  |   while l ≤ r and A[l] < pivot do
6  |   |   l++;
7  |   end
8  |   while l ≤ r and A[r] > pivot do
9  |   |   r- -;
10 |   end
11 |   if l ≤ r then
12 |   |   swap(A[l], A[r]);
13 |   |   l++; r- -;
14 |   end
15 end
16 swap(A[l], A[p]);
17 return l;
```



In-place sorting in Java

```
<K> void quickSortInPlace(K[]S,Comparator<K>comp,int a,int b){
    if (a >= b) return;
    int left = a;
    int right = b-1;
    K pivot = S[b];
    K temp;
    while (left <= right) {
      while (left<=right && comp.compare(S[left], pivot)<0) left++;
      while (left<=right && comp.compare(S[right], pivot)>0)
          right--;
      if (left <= right) {
        temp=S[left]; S[left]=S[right]; S[right]=temp;
        left++; right--;
      }
    }
    temp=S[left]; S[left]=S[b]; S[b]=temp;
    quickSortInPlace(S, comp, a, left - 1);
    quickSortInPlace(S, comp, left + 1, b);
  }
```

- An execution of quick-sort is depicted by a binary tree
- Each node represents a recursive call of quick-sort and stores
- Unsorted sequence before the execution and its pivot
- Sorted sequence at the end of the execution
- The root is the initial call
- The leaves are calls on subsequences of size 0 or 1



Worst-case for Quick sort

- The worst case for quick-sort occurs when the input is sorted
  - One of L and G has size n - 1 and the other has size 0
- The running time is proportional to the sum of:

$$n + (n-1) + \cdots + 2 + 1 = n(n+1)/2 \tag{1}$$

- Thus, the worst-case running time of quick-sort is $O(n^2)$
- Can avoid this be randomize the pivot

Average-case for Quick sort

- Average case number of comparisons is $1.39N \log N$.
- Probabilistic guarantee against worst case
- Better than MergeSort because
  - less space is needed
  - cache locality

Insertion Sort

Algorithm 10: Insertion-Sort(A)

```
1 for j = 2 to A.length do
2 |   key = A[j]
3 |   i = j-1
4 |   while i > 0 and A[i] > key do
5 |   |   A[i+1] = A[i]
6 |   |   i = i-1
7 |   end
8 |   A[i+1] = key
9 end
```

6 5 3 1 8 7 2 4

Best case running time is $O(n)$

- Let's focus on the number of comparisons.
- Checking number of all the 'operations' is more complicated.
- The best case is when A is already sorted. i.e.,

$$A[j-1] \leq A[j], \qquad (\text{for } j = 2, 3, \ldots, n)$$

- As this ensures that the condition of the *While* statement,

$$A[i] > key$$

  is false for $j = 2, 3, \ldots, n$,
- Inside the while loop there is only one comparison.
- The for loop checks n-1 elements

Thus, the number of comparison is

$$\sum_{j=2}^{n} 1 = n - 1.$$

Worst case number of comparisons

- In the worst case, the While condition will be tested for every possible value $i = j-1, j-2, \ldots, 1, 0$
- The cost for each while loop changes with j
- In total, it is

$$\sum_{j=2}^{n-1} j = \frac{n(n-1)}{2} - 1.$$

Average case number of comparisons

- For an "average" list, suppose that inside each while loop, we need to do half of the comparisons (and swaps),
- Hence the total number of comparisons is

$$\sum_{j=2}^{n-1} \sum_{j=2}^{n} j/2 = \frac{n(n-1)}{4} - 1.$$

- So the average running time of Insertion sort is $\Theta(N^2)$
- Any sorting algorithm that only swaps adjacent elements requires $\Omega(N^2)$ time because each swap removes only one inversion.
- If we are going to do better than $O(N^2)$, we are going to have to fix more than one inversion at a time
- How can we fix more than one inversion?
- Move the elements far away with each swap

Shell sort

- Named after Donald Shell – inventor of the algorithm
- Running time is $O(N^x)$, where x = 6/5, 5/4, 4/3, . . . , or 2, depending on "increment sequence"
- Shell sort uses repeated insertion sorts on selected subarrays of the larger array being sorted
- Multiple passes with changing subarrays

Shell Sort Example

Input	4	3	2	1

Shell Sort Example

Input	4	3	2	1
Two arrays with spacing=2	4	3	2	1

Shell Sort Example

Input	4	3	2	1
Two arrays with spacing=2	4	3	2	1
Sort the red array	2	3	4	1

Shell Sort Example

Input	4	3	2	1
Two arrays with spacing=2	4	3	2	1
Sort the red array	2	3	4	1
Sort the black array	2	1	4	3

Shell Sort Example

Input	4	3	2	1
Two arrays with spacing=2	4	3	2	1
Sort the red array	2	3	4	1
Sort the black array	2	1	4	3
Reduce the spacing =1	2	1	4	3
inverse 2/1	1	2	4	3

Shell Sort Example

Input	4	3	2	1
Two arrays with spacing=2	4	3	2	1
Sort the red array	2	3	4	1
Sort the black array	2	1	4	3
Reduce the spacing =1	2	1	4	3
inverse 2/1	1	2	4	3
inverse 4/3	1	2	4	3

Shell Sort Example

Input	4	3	2	1
Two arrays with spacing=2	4	3	2	1
Sort the red array	2	3	4	1
Sort the black array	2	1	4	3
Reduce the spacing =1	2	1	4	3
inverse 2/1	1	2	4	3
inverse 4/3	1	2	4	3

- number of inversions in shell sort: 4
- number of inversions in insertion sort: 1+2+3=6

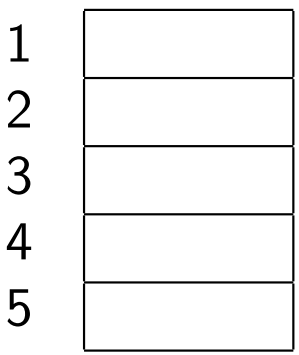
Intuition of Shell Sort

- Insertion sort runs fast on nearly sorted sequences
- Immediate termination when proper spot is found
- Do several passes of Insertion sort on different subsequences of elements
- The subsequences stay sorted from pass to pass

BucketSort, BinSort, CountingSort

Motivating example: If all values to be sorted are known to be between 1 and B

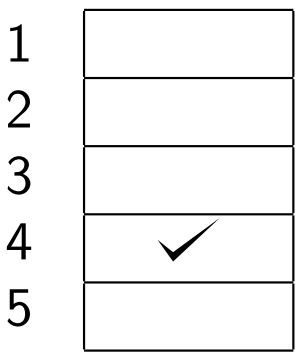
- create an array count of size B,
- increment counts while traversing the input,
- and finally output the result.
- e.g., Input=(4, 5, 1, 3, 5)
- B is 5. create 5 buckets using array:



BucketSort, BinSort, CountingSort

Motivating example: If all values to be sorted are known to be between 1 and B

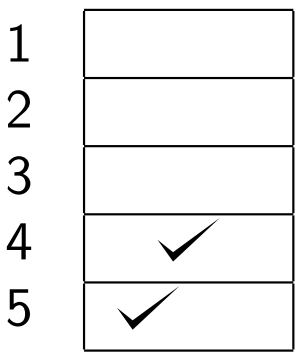
- create an array count of size B,
- increment counts while traversing the input,
- and finally output the result.
- e.g., Input=(4, 5, 1, 3, 5)
- B is 5. create 5 buckets using array:



BucketSort, BinSort, CountingSort

Motivating example: If all values to be sorted are known to be between 1 and B

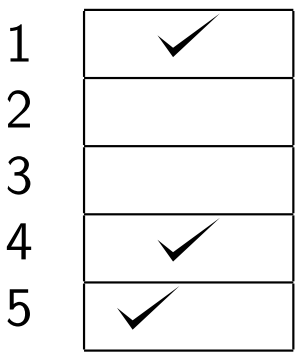
- create an array count of size B,
- increment counts while traversing the input,
- and finally output the result.
- e.g., Input=(4, 5, 1, 3, 5)
- B is 5. create 5 buckets using array:



BucketSort, BinSort, CountingSort

Motivating example: If all values to be sorted are known to be between 1 and B

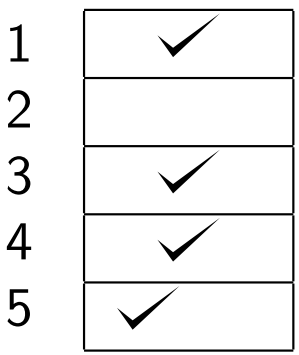
- create an array count of size B,
- increment counts while traversing the input,
- and finally output the result.
- e.g., Input=(4, 5, 1, 3, 5)
- B is 5. create 5 buckets using array:



BucketSort, BinSort, CountingSort

Motivating example: If all values to be sorted are known to be between 1 and B

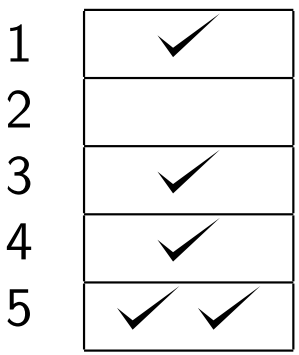
- create an array count of size B,
- increment counts while traversing the input,
- and finally output the result.
- e.g., Input=(4, 5, 1, 3, 5)
- B is 5. create 5 buckets using array:



BucketSort, BinSort, CountingSort

Motivating example: If all values to be sorted are known to be between 1 and B

- create an array count of size B,
- increment counts while traversing the input,
- and finally output the result.
- e.g., Input=(4, 5, 1, 3, 5)
- B is 5. create 5 buckets using array:



- We can extend the idea to sort real numbers
- Bucket can contain distinct items

Time Complexity

- What is the complexity?

Time Complexity

- What is the complexity? $O(n + B)$
- Depending on B
  - What if...
  - B is a constant
  - B is proportional to n
  - B is a very large constant (2^{32})

Fixing impracticality: RadixSort

- Radix Sort:generalization of Bucket Sort for large integer keys.
- Origins go back to the 1890 census.
- Radix= "The base of a number system"
- We use 10 for convenience, but could be anything
- Ideas:
  - Bucket Sort on one digit at a time
  - After k-th sort, the last k digits are sorted
  - Set number of buckets: B = radix.

First pass

- Input: 478, 537, 9, 721, 3, 38, 123, 67

	0	1	2	3	4	5	6	7	8	9
Sort on 1's										

First pass

- Input: 478, 537, 9, 721, 3, 38, 123, 67

	0	1	2	3	4	5	6	7	8	9
Sort on 1's									478	

First pass

- Input: 478, 537, 9, 721, 3, 38, 123, 67

	0	1	2	3	4	5	6	7	8	9
Sort on 1's								537	478	

First pass

- Input: 478, 537, 9, 721, 3, 38, 123, 67

	0	1	2	3	4	5	6	7	8	9
Sort on 1's								537	478	9

First pass

- Input: 478, 537, 9, 721, 3, 38, 123, 67

	0	1	2	3	4	5	6	7	8	9
Sort on 1's		721						537	478	9

First pass

- Input: 478, 537, 9, 721, 3, 38, 123, 67

	0	1	2	3	4	5	6	7	8	9
Sort on 1's		721		3				537	478	9

First pass

- Input: 478, 537, 9, 721, 3, 38, 123, 67

	0	1	2	3	4	5	6	7	8	9
Sort on 1's		721		3				537	478 38	9

First pass

- Input: 478, 537, 9, 721, 3, 38, 123, 67

	0	1	2	3	4	5	6	7	8	9
Sort on 1's		721		3 123				537	478 38	9

First pass

- Input: 478, 537, 9, 721, 3, 38, 123, 67

	0	1	2	3	4	5	6	7	8	9
Sort on 1's		721		3 123				537 67	478 38	9

First pass

- Input: 478, 537, 9, 721, 3, 38, 123, 67

	0	1	2	3	4	5	6	7	8	9
Sort on 1's		721		3 123				537 67	478 38	9

- Output of first pass: 721, 3, 123, 537, 67, 478, 38, 9
- used as the input of second pass

Second pass

- Input: 721, 3, 123, 537, 67, 478, 38, 9

	0	1	2	3	4	5	6	7	8	9
Sort on 10's										

Second pass

- Input: 721, 3, 123, 537, 67, 478, 38, 9

	0	1	2	3	4	5	6	7	8	9
Sort on 10's			721							

Second pass

- Input: 721, 3, 123, 537, 67, 478, 38, 9

	0	1	2	3	4	5	6	7	8	9
Sort on 10's	03		721							

Second pass

- Input: 721, 3, 123, 537, 67, 478, 38, 9

	0	1	2	3	4	5	6	7	8	9
Sort on 10's	03		721 123							

Second pass

- Input: 721, 3, 123, 537, 67, 478, 38, 9

	0	1	2	3	4	5	6	7	8	9
Sort on 10's	03		721 123	537						

Second pass

- Input: 721, 3, 123, 537, 67, 478, 38, 9

	0	1	2	3	4	5	6	7	8	9
Sort on 10's	03		721 123	537			67			

Second pass

- Input: 721, 3, 123, 537, 67, 478, 38, 9

	0	1	2	3	4	5	6	7	8	9
Sort on 10's	03		721 123	537			67	478		

Second pass

- Input: 721, 3, 123, 537, 67, 478, 38, 9

	0	1	2	3	4	5	6	7	8	9
Sort on 10's	03		721 123	537 38			67	478		

Second pass

- Input: 721, 3, 123, 537, 67, 478, 38, 9

	0	1	2	3	4	5	6	7	8	9
Sort on 10's	03 09		721 123	537 38			67	478		

Second pass

- Input: 721, 3, 123, 537, 67, 478, 38, 9

	0	1	2	3	4	5	6	7	8	9
Sort on 10's	03 09		721 123	537 38			67	478		

- Output of second pass: 3,9,721, 123, 537, 38, 67, 478
- used as the input of second pass

Third pass

- Input: 3,9,721, 123, 537, 38, 67, 478

	0	1	2	3	4	5	6	7	8	9
Sort on 100's										

- Output: 3, 9, 38, 67, 123, 537, 721.

Third pass

- Input: 3,9,721, 123, 537, 38, 67, 478

	0	1	2	3	4	5	6	7	8	9
Sort on 100's	003									

- Output: 3, 9, 38, 67, 123, 537, 721.

Third pass

- Input: 3,9,721, 123, 537, 38, 67, 478

	0	1	2	3	4	5	6	7	8	9
Sort on 100's	003 009									

- Output: 3, 9, 38, 67, 123, 537, 721.

Third pass

- Input: 3,9,721, 123, 537, 38, 67, 478

	0	1	2	3	4	5	6	7	8	9
Sort on 100's	003 009							721		

- Output: 3, 9, 38, 67, 123, 537, 721.

Third pass

- Input: 3,9,721, 123, 537, 38, 67, 478

	0	1	2	3	4	5	6	7	8	9
Sort on 100's	003 009	123						721		

- Output: 3, 9, 38, 67, 123, 537, 721.

Third pass

- Input: 3,9,721, 123, 537, 38, 67, 478

	0	1	2	3	4	5	6	7	8	9
Sort on 100's	003 009	123				537		721		

- Output: 3, 9, 38, 67, 123, 537, 721.

Third pass

- Input: 3,9,721, 123, 537, 38, 67, 478

	0	1	2	3	4	5	6	7	8	9
Sort on 100's	003 009 038	123				537		721		

- Output: 3, 9, 38, 67, 123, 537, 721.

Third pass

- Input: 3,9,721, 123, 537, 38, 67, 478

	0	1	2	3	4	5	6	7	8	9
Sort on 100's	003 009 038 067	123				537		721		

- Output: 3, 9, 38, 67, 123, 537, 721.

Third pass

- Input: 3,9,721, 123, 537, 38, 67, 478

	0	1	2	3	4	5	6	7	8	9
Sort on 100's	003 009 038 067	123			478	537		721		

- Output: 3, 9, 38, 67, 123, 537, 721.

Complexity of Radix sort

- Input size, n
- In our example,
  - Number of buckets, B $= 10$
  - Maximum value, three digits, $M < 10^3$
  - Number of passes, $P = \log_B M = 3$
- Run time proportional to

$$(B + n)P = (B + n) \log_B M \tag{2}$$

- In practice, B should be much larger than 10, so that P is a small number.

Comparison of sorting algorithms

	Method	Time Best	Worst	Avg	In-place	Stable
Comparison-based	Selection	n^2	n^2	n^2	$O(1)$	No
	Insertion	n	n^2	n^2	$O(1)$	Yes
	Shell			$n^{6/5}$	$O(1)$	No
	HeapSort	$n \log n$	$n \log n$	$n \log n$	$O(1)$	No
	Merge	$n \log n$	$n \log n$	$n \log n$	$O(n)$	Yes
	Quick	$n \log n$	n^2	$n \log n$	$O(1)$.	No
Counting-based	Bucket			O(n+B)	O(B)	Yes
	Radix			O(n+B)	O(B)	Yes

- Quick sort is often used in practice. (e.g., in Java)
  - It is in-place sorting. Save space compared with mergeSort.
- merge Sort: good for external memory

Takeaways

- Evaluation of sorting algorithms: Time, space, stability, data dependency, application dependency
- Quick sort. Divide and conquer. Time complexity. Worst case and average case. Its comparison with merge sort.
- Shell sort.
- Bucket and Radix sort.
- Readings: Goodrich et al. P532-562.
- YouTube video for 15 sorting algorithms in 6 minutes